

题目描述

题解

提交记录

LCR 004. 只出现一次的数字 II

已解答

中等

相关标签

相关企业

Aa

给你一个整数数组 `nums`，除某个元素仅出现一次外，其余每个元素都恰出现三次。请你找出并返回那个只出现了一次的元素。

示例 1:

输入: `nums = [2,2,3,2]`

输出: `3`

示例 2:

输入: `nums = [0,1,0,1,0,1,100]`

输出: `100`

C++

智能模式

```
1 class Solution {
2 public:
3     int singleNumber(vector<int>& nums) {
4         unordered_map<int,int> map;
5         for(int num:nums){
6             ++map[num];
7         }
8         int ans = 0;
9         for(auto [num,occ]:map){
10             if(occ == 1){
11                 ans = num;
12                 break;
13             }
14         }
15         return ans;
16     }
17 };
```

题目描述

题解

提交记录

LCR 005. 最大单词长度乘积

已解答

中等

相关标签

相关企业

Aa

给定一个字符串数组 `words`，请计算当两个字符串 `words[i]` 和 `words[j]` 不包含相同字符时，它们长度的乘积的最大值。假设字符串中只包含英语的小写字母。如果没有不包含相同字符的一对字符串，返回 0。

示例 1:

输入: `words = ["abcw","baz","foo","bar","fxyz","abcdef"]`

输出: `16`

解释: 这两个单词为 "abcw", "fxyz"。它们不包含相同字符，且长度的乘积最大。

示例 2:

输入: `words = ["a","ab","abc","d","cd","bcd","abcd"]`

输出: `4`

解释: 这两个单词为 "ab", "cd"。

示例 3:

C++

智能模式

```
1 class Solution {
2 public:
3     int maxProduct(vector<string>& words) {
4         int length = words.size();
5         vector<int> masks (length);
6         int i = 0;
7         for(string word:words){
8             int wordLength = word.size();
9             for(int j = 0;j<wordLength;j++){
10                 //word[j] - 'a'; if word[j] = b 'b' - 'a' ==1; 1<1 -> 0001 -> 0010
11                 masks[i] |= 1 << (word[j] - 'a');
12             }
13             i++;
14         }
15
16         int maxAns = 0;
17         for(int i = 0;i<length;i++){
18             for(int j = i+1;j<length;j++){
19                 if((masks[i]&masks[j]) == 0){
20                     maxAns = max(maxAns,int(words[i].size()*words[j].size()));
21                 }
22             }
23         }
24         return maxAns;
25     }
26 };
```

题目描述

题解

提交记录

LCR 006. 两数之和 II - 输入有序数组

已解答

简单

相关标签

相关企业

Aa

给定一个已按照升序排列的整数数组 `numbers`，请你从数组中找出两个数满足相加之和等于目标数 `target`。

函数应该以长度为 2 的整数数组的形式返回这两个数的下标值。`numbers` 的下标从 0 开始计数，所以答案数组应当满足 `0 <= answer[0] < answer[1] < numbers.length`。

假设数组中存在且只存在一对符合条件的数字，同时一个数字不能使用两次。

示例 1:

输入: `numbers = [1,2,4,6,10], target = 8`

输出: `[1,3]`

解释: 2 与 6 之和等于目标数 8。因此 index1 = 1, index2 = 3。

示例 2:

C++

智能模式

```
1 class Solution {
2 public:
3     vector<int> twoSum(vector<int>& numbers, int target) {
4         //申请一个哈希表,key-value 整数-下标
5         unordered_map<int,int>map;
6         //For循环遍历整个数组
7         for(int i=0;i<numbers.size();i++){
8             //求出当前需要的元素,用目标减去当前元素
9             int need = target - numbers[i];
10            //找到当前需求的元素后,去哈希表中寻找是否有此元素
11            if(map.find(need) != map.end()){
12                //如果有,返回需求元素下标和当前元素下标
13                return {map[need],i};
14            }
15            //如果没有 把当前元素当作key 下标当作value 存入哈希表
16            map[numbers[i]] = i;
17        }
18        return {};
```